# DIABETES PREDICTION

Documentation

Prepared by : Rajshree Sharma
Mail : shalinionmmail15@gmail.com

## Project Overview

This is a ML-Based Diabetes Prediction web application designed to predict selected patient's health after they input their health information for estimating the likelihood of diabetes. The project uses a supervised Random Forest classification model trained on a diabetes health excel sheet that is used as dataset. A Flask web interface allows a user to enter health values and receive a prediction together with an estimated probability.

## Project Objectives

This process has some objectives:

- To build a machine-learning model for diabetes prediction.
- Evaluate the trained model using standard classification metrics.
- Save the trained model.
- To Provide an interactive web application.
- Deploy the application for public access to the web service.

## Software Requirements

- Python 3
- pandas
- numpy
- scikit-learn
- joblib
- Flask
- Gunicorn for production deployment

# Project Related Codes and Links-

**Model Training Code File -**
https://drive.google.com/file/d/1ZvVuArkOF1lJpQCekAhtNwjCeABi5LSH/view?usp=sharing

**Web Application Code File-**
https://drive.google.com/file/d/1Z5nmSGVKrrQ8FUiwU4GpNSkKeOL_Pp-t/view?usp=sharing

**GitHub Repo File - (Contains Live Link Of Website and all files required)**

https://github.com/rajshreeongitnow02/AI-Based-Diabates-Prediction/tree/main

**Live Access To The Website -**

https://ai-based-diabates-prediction.onrender.com

# Dataset Description

The training script loads the dataset from Dataset/diabetes_binary_health (1).csv. The target column is named Diabetes. All remaining columns are treated as input features.

| FEATURES | DATA TYPE | DATA |
|---|---|---|
| Age | Numeric | Patient age |
| BMI | Numeric | Body Mass Index |
| Physical Activity | Binary / categorical | Physical activity level encoded as 0 or 1 |
| Blood Pressure | Numeric | Blood pressure value |
| Cholesterol | Binary / categorical | Cholesterol category encoded as 0 or 1 |
| Glucose | Numeric | Glucose level |
| Diabetes | Target | Diabetes class used for supervised learning |

# Data Preprocessing

The preprocessing pipeline in the supplied training script consists of the following steps:

1. Load the CSV dataset using pandas.
2. Separate the target column Diabetes from the input features.
3. Apply Simple Imputer with the median strategy to handle missing feature values.
4. Convert the imputed feature matrix back into a pandas Data Frame while preserving feature names.
5. Split the data into training and testing sets using an 80:20 ratio.
6. Use stratification during the split so that the class distribution is maintained as closely as possible.
7. Use the same saved imputer later in the Flask application before making a prediction.
8. The imputer is fitted on the training-data preparation stage and saved as diabetes_imputer.pkl.
9. The trained Random Forest model is saved as diabetes_random_forest.

# Model Evaluation and Results

## System Workflow

**User opens the Flask web application.**

|

**User enters the six required health inputs.**

|

**Flask server receives submitted form.**

|

**Input values are converted into the expected numeric representation.**

|

**Saved imputer handles missing values according to the training preprocessing.**

|

**Random Forest model generates the predicted class.**

|

**predict_proba( ) is used to calculate the estimated probability for the positive class.**

|

**The result and probability are displayed on the web page.**

# Training Model

Execution of code ‘train_model’ from VS code-

```
Problems  Output  Debug Console  Terminal  Ports
[Running] python -u "c:\Users\Aditya's Gaming RIG\OneDrive\Desktop\didi's file\Projects\AI-Based-Dia
Dataset loaded successfully!
Shape:  (2000, 7)

First 5 rows:
   Age   BMI  PhysicalActivity  BloodPressure  Cholesterol  Glucose  Diabetes
0   23  23.4                 1          126.0          0.0      NaN         0
1   66  32.6                 1          156.0          0.0     90.0         0
2   59  27.7                 0          120.0          NaN    133.0         0
3   45  25.9                 0          128.0          0.0    104.0         0
4   45  26.9                 1          134.0          1.0    134.0         0

Features:
['Age', 'BMI', 'PhysicalActivity', 'BloodPressure', 'Cholesterol', 'Glucose']
```

```
Missing values handled successfully.

Training samples: 1600
Testing samples: 400

Random Forest model trained successfully!

 MODEL EVALUATION
Accuracy  : 0.9050
Precision : 0.5000
Recall    : 0.4474
F1-Score  : 0.4722
ROC-AUC   : 0.9057
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.95      0.95       362
           1       0.50      0.45      0.47        38

    accuracy                           0.91       400
   macro avg       0.72      0.70      0.71       400
weighted avg       0.90      0.91      0.90       400
```

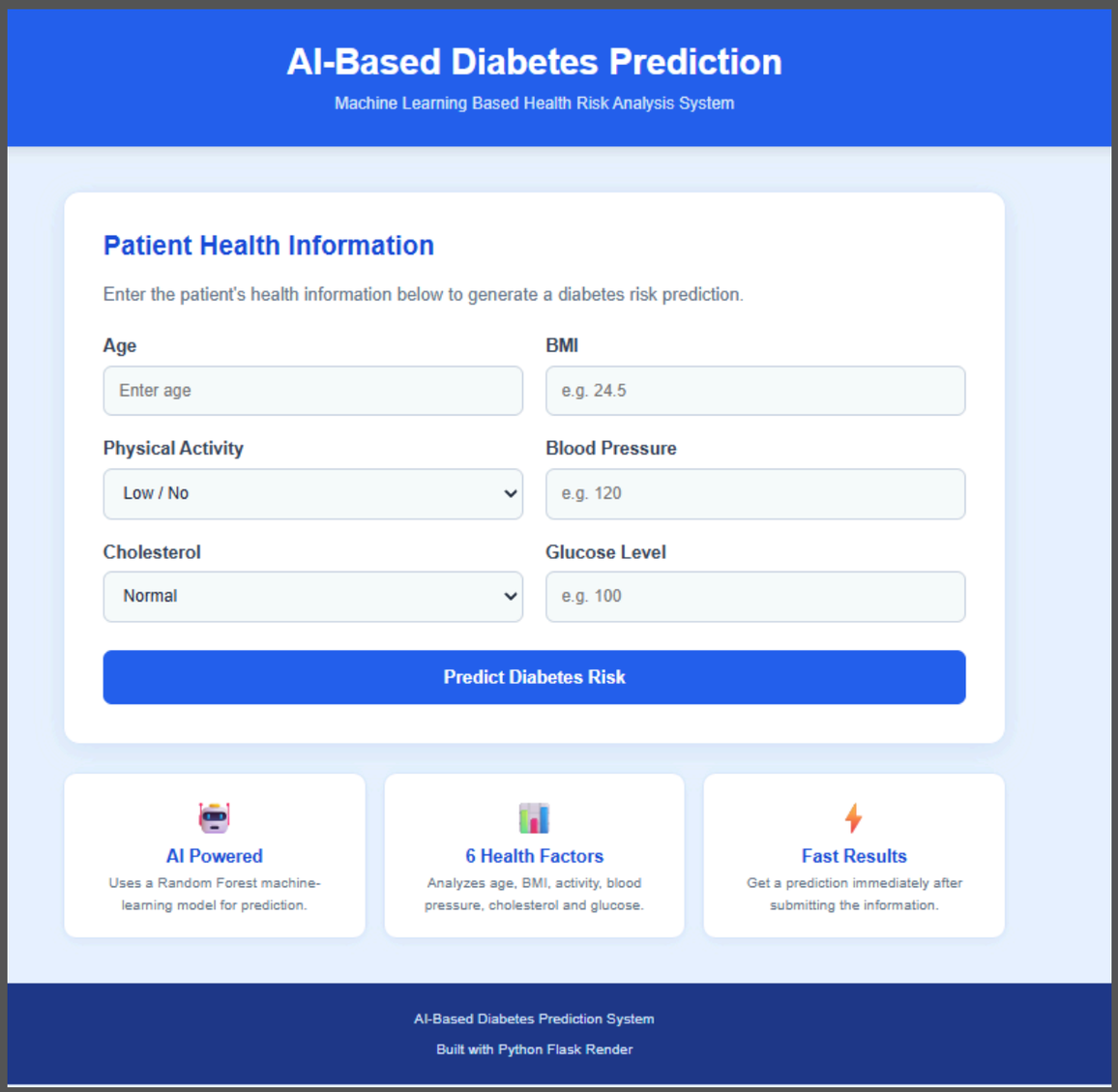
```
 FEATURE IMPORTANCE
          Feature  Importance
5         Glucose    0.336568
0             Age    0.211621
3   BloodPressure    0.188192
1             BMI    0.169259
4     Cholesterol    0.076443
2  PhysicalActivity    0.017917

Model saved successfully!
Files created inside the model folder.
```

# Results-

A basic web application showing near to accurate prediction of whether a person is diabetic or not based on the model training.

Execution of ‘app.py’ code-



# Evaluation Metrics

| Metric | Value from training run | Purpose |
|---|---|---|
| **Accuracy** | **0.9050 (90.50%)** | Overall proportion of correct predictions |
| **Precision** | **0.5000 (50.00%)** | Proportion of positive predictions that are correct |
| **Recall** | **0.4474 (44.74%)** | Proportion of actual positive cases identified |
| **F1-Score** | **0.4722 (47.22%)** | Balance between precision and recall |
| **ROC-AUC** | **0.9057 (90.57%)** | Measures ranking/discrimination performance across thresholds |

The above data is collected from VS code after running the 'train_model' code -

```
Random Forest model trained successfully!

 MODEL EVALUATION
Accuracy  : 0.9050
Precision : 0.5000
Recall    : 0.4474
F1-Score  : 0.4722
ROC-AUC   : 0.9057

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.95      0.95       362
           1       0.50      0.45      0.47        38

    accuracy                           0.91       400
   macro avg       0.72      0.70      0.71       400
weighted avg       0.90      0.91      0.90       400
```